Advanced CI/CD with Jenkins on Ubuntu WSL2

Student lab | Foundations and Efficient Operation of Modern Infrastructure | 2026/27/1

Scenario: You are a DevOps engineer working on a small inventory service for a network operations team. Each code change must be checked by automated unit tests, packaged as a non-root Docker image, verified with HTTP smoke tests, and deployed only if the earlier stages succeed. Jenkins itself runs directly as an Ubuntu service in WSL2, not inside Docker.

Level: intermediate/advanced | Estimated classroom scope: 1 extended lab plus independent completion | Submission: Git repository, Jenkins evidence, and short engineering report.

1. Learning outcomes and architecture

By the end, you will: configure WSL2 with systemd; install Jenkins LTS using apt; practice Git-based CI; write and interpret pytest tests; build a Docker image using a non-root account; implement a declarative Jenkinsfile with a failure gate; verify deployed HTTP endpoints; and diagnose deliberately broken builds.

```
Windows 11
  +-- WSL2 Ubuntu 24.04 (systemd)
        +-- Jenkins service :8080 (native apt package)
        +-- Git + Python3 + pytest + Docker CLI
        +-- Docker Engine (native OR Docker Desktop WSL integration)
              +-- inventory-candidate :5051 (temporary smoke test)
              +-- inventory-live      :5050 (local deployment)
GitHub (main branch) --> Jenkins checkout --> tests --> image --> smoke --> deploy
```

Safety: work in an isolated personal WSL environment. Granting the Jenkins account access to the Docker socket is effectively root-level access. Do not expose Jenkins or the application to the public internet. Never commit credentials or tokens.

2. Prerequisites and WSL2 preparation

From PowerShell (Administrator), install WSL if not yet installed. Reboot if Windows requests it. On an existing WSL installation, use wsl --update and confirm Ubuntu is WSL version 2.

```
wsl --install -d Ubuntu-24.04
wsl --list --verbose
wsl --update
```

Inside the Ubuntu terminal, use a Linux home directory (for example, ~/projects) rather than /mnt/c. Enable systemd if it is not already enabled:

```
sudo mkdir -p /etc
printf '[boot]\nsystemd=true\n' | sudo tee /etc/wsl.conf
```

From PowerShell run wsl --shutdown, reopen Ubuntu and confirm:

```
ps -p 1 -o comm=   # expected: systemd
sudo apt update
sudo apt install -y git curl wget ca-certificates fontconfig \
  openjdk-21-jre python3 python3-venv python3-pip
```

Docker option A: install Docker Desktop for Windows, enable WSL integration for this Ubuntu distribution, then use the Docker CLI inside Ubuntu. Docker option B: install Docker Engine directly in Ubuntu following the official Docker Ubuntu instructions and enable its systemd service. Choose one option, not both. Confirm:

```
docker version
docker run --rm hello-world
```

3. Install Jenkins natively in Ubuntu WSL2

These commands use the current Jenkins LTS apt signing key (2026). Jenkins requires a supported Java runtime; this lab uses Java 21.

```
sudo install -d -m 0755 /etc/apt/keyrings
sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \
  https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key
echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc] \
  https://pkg.jenkins.io/debian-stable binary/" | \
  sudo tee /etc/apt/sources.list.d/jenkins.list >/dev/null
sudo apt update
sudo apt install -y jenkins
sudo systemctl enable --now jenkins
sudo systemctl status jenkins --no-pager
```

Open http://localhost:8080 from Windows. Retrieve the first-time administrator password using:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Install Suggested Plugins and create a named admin account. Verify the Pipeline, Git and JUnit plugins are available in Manage Jenkins / Plugins. Avoid using the shared built-in admin credentials in screenshots or submissions.

For this local training setup only, allow the Jenkins service account to use Docker and restart Jenkins:

```
sudo usermod -aG docker jenkins
sudo systemctl restart jenkins
sudo -u jenkins docker ps
```

If that last command fails, inspect docker version, the Docker daemon, socket access, the service logs (sudo journalctl -u jenkins -n 50) and the Jenkins user groups. With Docker Desktop integration, verify its WSL integration settings. If port 8080 conflicts with another service, see the official Jenkins Linux installation guidance for a systemd override.

4. Get the starter project and understand the API

Extract Jenkins_WSL_Student_Starter.zip into ~/projects/jenkins_wsl_lab_starter. The ZIP contains: app/main.py, tests/test_api.py, requirements.txt, requirements-dev.txt, Dockerfile, Jenkinsfile, .gitignore, .dockerignore, and README.md.

The service exposes GET /, GET /health, GET /api/items, and GET /api/items/<id>. Valid items return JSON. An unknown ID returns HTTP 404. All requests run on the Docker container internal port 5000.

Task A - run the quality checks locally

```
cd ~/projects/jenkins_wsl_lab_starter
python3 -m venv .venv
.venv/bin/pip install -r requirements-dev.txt
.venv/bin/python -m pytest -v
docker build -t inventory-api:local .
docker run -d --name inventory-manual \
  -p 127.0.0.1:5050:5000 inventory-api:local
curl -i http://127.0.0.1:5050/health
curl -i http://127.0.0.1:5050/api/items/999
docker rm -f inventory-manual
```

Checkpoint: explain why 127.0.0.1:5050 on the host is not the same address as port 5000 inside the container; identify whether the image runs as root or a non-root UID.

5. Git and Jenkins pipeline-as-code

Create a GitHub repository, for example inventory-ci-lab. Make it public for easier checkout or keep it private and configure a scoped Jenkins credential. Do not enter an account password as a Git credential; use SSH or a suitable token when needed.

```
git init -b main
git add .
git commit -m "Initial inventory API with pipeline"
git remote add origin https://github.com/YOUR_USER/inventory-ci-lab.git
git push -u origin main
```

In Jenkins: New Item > Pipeline; name: InventoryPipeline. Choose Pipeline script from SCM; SCM = Git; repository URL = your GitHub repository; branch = */main; script path = Jenkinsfile. Supply Jenkins credentials only when required. Trigger the first build manually with Build Now.

Read the included Jenkinsfile carefully. Its stages are Checkout > Unit tests > Build image > Container smoke test > Deploy locally. Unit tests publish a JUnit XML report, and the smoke-test container is deleted even if the build fails. The final deployment can be opened at http://localhost:5050/health.

Important: deploying the live container removes the old live container, so this baseline pipeline has a brief outage and no automatic rollback. Investigating and improving these limitations is part of the harder challenge below.

6. Required engineering challenges (work individually)

Challenge 1 - extend the API and unit tests (20 points)

Add GET /api/items/low-stock?threshold=N. Return items whose stock is strictly below N. Validate N as an integer in the range 0 to 100, returning HTTP 400 with a JSON error for invalid values. Create at least five new pytest cases including boundary values, bad input, and successful filtered results. Do not hardcode a successful result just to pass the test.

Challenge 2 - prove the CI failure gate (15 points)

Temporarily introduce a bug, such as returning the wrong health status, or intentionally make a unit test fail. Commit the change and rerun Jenkins. Capture the failed unit-test stage and show that the image build and deploy stages did not execute. Repair the bug in a new commit and capture the subsequent successful run. Include both commit IDs in your report.

Challenge 3 - improve deployment safety (20 points)

Implement one of these approaches and demonstrate it with evidence: (a) preserve and restore the previous working image/container when post-deployment validation fails; OR (b) implement blue/green traffic switching with a locally documented switch mechanism and a rollback procedure. Explain what happens to existing client requests during cutover. Document how you tested failure recovery.

Challenge 4 - quality and security checks (15 points)

Add a meaningful automated static check (for example, ruff check app tests) as a separate Jenkins stage. Harden the Docker setup: use a non-root application user, exclude secrets and local virtual environments from the image build context, and demonstrate that the final application runs without development dependencies. Do not put GitHub tokens in the Jenkinsfile.

Challenge 5 - design explanation and reproducibility (10 points)

Draw a simple architecture diagram and provide a clean README with setup, endpoints, testing, pipeline sequence and error handling. Demonstrate that a fresh clone can execute unit tests using only the declared requirements. Explain one risk associated with Docker socket access and one safer production alternative (e.g. a separate ephemeral build agent).

7. Optional stretch challenges (up to 20 bonus points)

(1) Implement a multibranch or pull-request workflow where feature branches run tests/builds but never deploy (10 points). (2) Add a reproducible dependency lock or a basic image vulnerability scan, publishing an artifact or report in Jenkins (10 points). Label bonus work clearly and provide screenshots or logs.

8. Evidence to submit and marking scheme

Submit: (i) a GitHub URL with complete source and Jenkinsfile; (ii) one PDF report of 3-5 pages containing environment versions, architecture, test outputs, Jenkins stage results, failure-and-fix proof, smoke tests, security observations and design decisions; (iii) at least four screenshots: a passed Jenkins pipeline, a failed test stage, JUnit results, and a live API response. Remove personal access tokens and passwords from all submissions.

Component	Points
Working WSL / Jenkins / Git / Docker baseline	20
Challenge 1 - API + tests	20
Challenge 2 - failure gate	15
Challenge 3 - safer deployment	20
Challenge 4 - lint / security	15
Challenge 5 - documentation / reasoning	10
TOTAL	100

9. Troubleshooting and instructor notes

Build stuck: check the Jenkins service and executor status. Checkout fails: verify the branch is main, repository URL, Git plugin and credentials. docker.sock permission denied: check daemon integration, group membership and restarted Jenkins service. Port conflict: run docker ps or ss -tlnp. Smoke test fails: inspect docker logs inventory-candidate, port mapping and the /health response. JUnit empty: ensure pytest is generating reports/pytest.xml. Missing Java: run java -version.

Instructor caution: do not use this baseline pipeline on shared university infrastructure. Docker access from Jenkins and local network bindings are training simplifications, not a production security design. A richer exercise could use a dedicated agent, secret store and guarded deployment environment.

10. Sources and adaptation note

This lab is a modernized, significantly extended teaching adaptation of the user-supplied Lab - Build a CI/CD Pipeline Using Jenkins (Cisco NetAcad, 2020-2021). The original uses a DEVASC VM, Docker-hosted Jenkins, freestyle build/test jobs and a basic pipeline. This version uses WSL2, native Jenkins, pipeline-as-code, a Flask inventory API, repeatable tests and deployment challenges.

Current installation references: Jenkins Linux installation; Jenkins LTS packages; WSL systemd setup; Docker Desktop WSL integration.